

Sprawozdanie: Eliminacja elementów zasłoniętych (BSP Tree)

Adam Boros
Kacper Warpechowski

10 maja 2026

1 Cel i zakres zadania

Celem projektu jest implementacja algorytmu **BSP Tree (Binary Space Partitioning)** do poprawnego rysowania zbioru trójkątów w przestrzeni 3D z uwzględnieniem wzajemnego zasłaniania. Scena jest pozbawiona systemowego, kosztownego Z-bufora. Obiekty są wypełnione jednolitym kolorem (solid rendering), a ich poszczególne definicje wczytywane są z zewnętrznego pliku tekstowego.

2 Środowisko uruchomieniowe

Projekt został napisany w języku **Python** przy wykorzystaniu powłoki `pygame-ce` do rysowania geometrii 2D oraz `numpy` do przyspieszenia operacji na macierzach. Aplikacja bazuje rdzennie na wirtualnej kamerze stworzonej w pierwszej części projektu, adaptując całe jej przestrzenne rzutowanie i sterowanie na wyższy model cieniowań. Do wczytywania sceny używany jest plik `scene.txt`.

3 Realizacja graficzna (opis algorytmiczny)

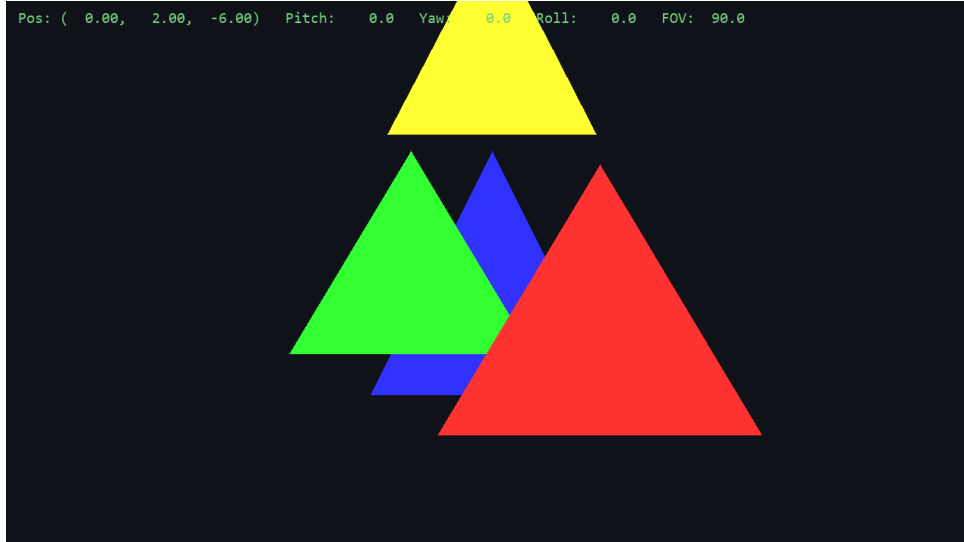
Program funkcjonuje opierając wymiary przestrzeni o informacje z pliku tekstowego, w którym każdy wiersz definiuje jeden trójkąt w formacie:

```
x1 y1 z1  x2 y2 z2  x3 y3 z3  R G B
```

Na bazie otrzymanych wielokątów budowane jest **drzewo BSP**. Pierwszy wczytany trójkąt stanowi płaszczyznę odniesienia podziału. Pozostałe ładowane struktury w klasyfikacji mogą zostać umiejscowione przed ową płaszczyzną, ukryte za nią lub, w bardzo niefortunnych kolizjach, podzielone geometrycznie na mniejsze części. W trakcie odświeżania algorytm przechodzi całe to drzewo posiłkując się aktualną pozycją obserwatora, wypuszczając listę poćwiartowanych figur prosto do narysowania w sekwencji od obiektu najdalszego od kamery do najbliższego obiektywowi (tzw. Algorytm Malarza). Przez to obiekty ułożone faktycznie na przodzie nakrywają barwnymi pikselami struktury, które narysowane zostały chwilę wcześniej gdzieś z tyłu w tle.

4 Geometria sceny testowej

Dla zbadania fizyki nakładania się pikseli, wykreowano sztuczny układ czterech osobnych rzutnych trójkątów (czerwony, zielony, niebieski i ukośny żółty "daszek"), ułożonych szeregowo jeden za drugim.



Rysunek 1: Scena 3D widziana frontalnie, tuż po uruchomieniu aplikacji.



Rysunek 2: Układ wygenerowanych trójkątów na siatce współrzędnych - rzut na ułożenie obiektów w głębi sceny.

Zrzuty zamieszczone powyżej (Rysunek 1 oraz Rysunek 2) pełnią formę informacyjną wobec struktury mapy, pokazując tylko konstrukcję oraz ułożenie fizyczne użytych przedmiotów w bezpiecznej odległości i kolejności od Z. Pozwoli to z łatwością ocenić niżej umiejscowione zadania testowe.

5 Testy działania

Kluczowymi dowodami dla eliminacji zasłoniętej powłoką graficznej okluzji, były obroty kamerowe łamiące utarty system naturalnych położeń kamery do przodu na korzyść wpadania obiektów na siebie podczas perspektyw bocznych.

5.1 Test 1: Przysłonięcia przy rotacji kamery w lewo

Czynność: Przesunięcie i agresywny obrót obiektywu strzałkami na scenę pod lewym kątem.

Oczekiwany efekt: Skrajnie najbardziej niebieski trójkąt w osi całej sceny po odwróceniu kamery pod nowym wkłęśnięciem kątowym staje się wielokątem z przodu i poprawnie przysłania wycinek z obiektu uprzednio widywanego na przodzie.

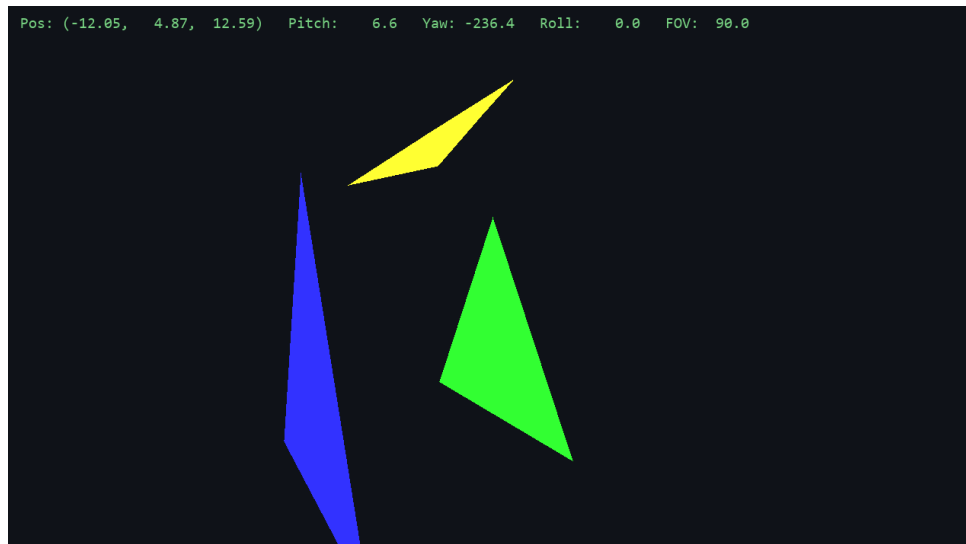


Rysunek 3: Niebieski trójkąt prawidłowo przysłania widok z tyłu, blokując render elementu zielonego w osi wizualnej kamery.

5.2 Test 2: Przysłonięcia przy rotacji kamery w prawo

Czynność: Odwrócenie rzutu i przelot ogniskową prosto w prawy bok geometryczny.

Oczekiwany efekt: Natychmiastowe przetasowanie na odległości w pod-drzewach malarza odcinające dominującą do tej pory moc obiektu czerwonego, na korzyść zielonego trójkąta od którego teraz odbijają się pierwsze siatki wizji.



Rysunek 4: Zamierzone zakrycie centralnego czerwonego przedmiotu odświeżającą zmianą perspektywy z prawej.

5.3 Test 3: Okluzja elementu ukośnego (pochyłego)

Czynność: Odejdźcie do góry ponad konstrukcję na widok żółtego wzdłużnego elementu.

Oczekiwany efekt: Pokrywający pozostałe wierzchołki trójkąt rygorystycznie ucina środek widocznemu czerwonemu wycinkowi poniżej przy czym algorytm nie rysuje fałszywych wpieć trójkąta pod obrzary żółtego.



Rysunek 5: Prawidłowe odcinanie rzutu brył pod pochylą, skrajnym obiektem.

6 Wnioski końcowe

Drzewo binarne BSP bardzo dobrze uporało się z postawionym zadaniem. Należyte zaimplementowanie przechodzenia algorytmu umożliwiło sortowanie wielowierzchołkowych map generując rzut całkowicie bez wyliczania fizycznych obciążeń pamięci per Z, co udo-

wadnia siłę okluzji opartej na węzłowej przestrzeni odcinającej defekty graficzne (artefakty widoczności krawędzi obiektów w oddali).